

Q1. Disk Storage Allocation Methods

Describe different allocation methods in disk storage. Explain with a neat sketch. (16 marks)

When files are stored on disk, the operating system must decide how disk blocks are allocated to a file. The goal is fast access, efficient space usage, and the ability for files to grow and shrink easily. The three classical allocation methods are contiguous allocation, linked allocation, and indexed allocation.

1. Contiguous Allocation

Each file occupies a set of contiguous (consecutive) blocks on the disk. The directory entry for a file stores only the starting block number and the length (number of blocks) of the file.

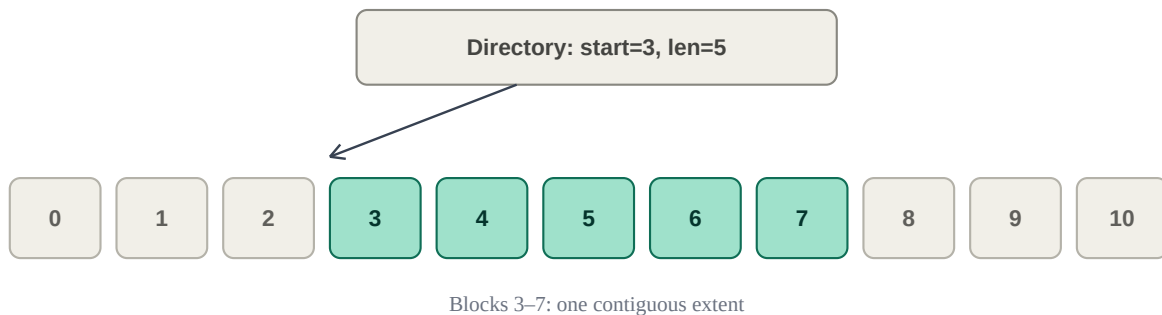


Fig 1.1 – Contiguous allocation: directory points to a start block, file occupies a run of consecutive blocks.

Advantages

- Excellent performance for both sequential and direct access, since block i of the file is simply $\text{start} + i$.
- Very little directory information is needed (just start address and length).

Disadvantages

- Suffers from external fragmentation as files are created and deleted, leaving scattered free holes.
- Requires the file size to be known in advance, which is often impossible.
- Growing a file may require relocating the entire file if the next block is already in use.

2. Linked Allocation

Each file is a linked list of disk blocks, which can be scattered anywhere on the disk. Every block stores the file data plus a pointer to the next block of the file. The directory holds only the addresses of the first and last blocks.

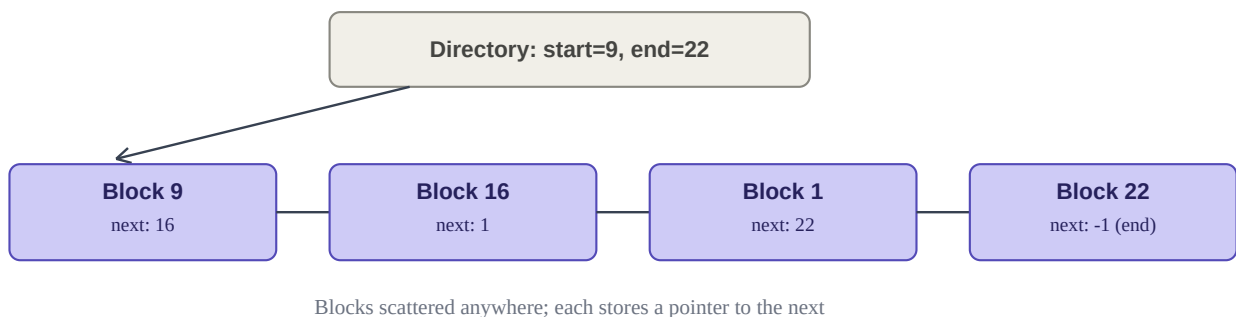


Fig 1.2 – Linked allocation: each block carries a pointer to the next block in the chain.

Advantages

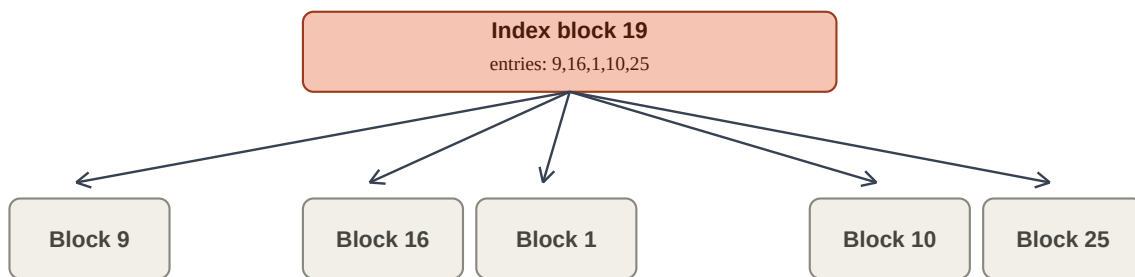
- No external fragmentation – any free block anywhere on disk can be used.
- Files can grow dynamically without declaring their size in advance.

Disadvantages

- No efficient direct/random access – reaching block i requires traversing the chain from the start.
- Space is wasted storing a pointer inside every block.
- Reliability risk: a single corrupted or lost pointer breaks access to the rest of the file (the FAT variant mitigates this by keeping the pointer table separately, in memory/on disk, instead of inside each block).

3. Indexed Allocation

All the block pointers for a file are collected into one place, called the index block. The directory entry stores the address of this index block, and the index block itself contains an array of pointers to the actual data blocks of the file.



Index block holds pointers to every data block of the file

Fig 1.3 – Indexed allocation: one index block holds pointers to every data block of the file.

Advantages

- Supports direct access – block i is found instantly via `index[i]`, with no chain traversal.
- No external fragmentation, and files can grow (subject to index block capacity).

Disadvantages

- Index block overhead is proportionally wasteful for very small files.
- A single index block limits maximum file size; solved in practice using:
- **Linked index blocks** – index blocks chained together for very large files.
- **Multilevel index** – an index block pointing to other index blocks (like a two-level page table).
- **Combined scheme (UNIX inode)** – a few direct pointers for small files plus single, double and triple indirect pointers for large files; this is the scheme used by traditional UNIX file systems.

Comparison Summary

Aspect	Contiguous	Linked	Indexed
Access type	Sequential + direct	Sequential only	Sequential + direct
External fragmentation	Yes	No	No
Space overhead	None	One pointer / block	One index block / file
File growth	Difficult	Easy	Easy (within index limits)
Reliability	Good	Weak (chain can break)	Good

Q2. File Sharing and File Protection

Discuss the mechanisms for file sharing and file protection in an operating system. Analyze how these mechanisms ensure security and consistency when multiple users access files concurrently.

Multi-user operating systems allow several users, or several processes belonging to the same user, to access the same file at the same time. This raises two related concerns: **sharing** – how the file system presents a shared file and keeps it usable by everyone, and **protection** – how the file system restricts access so that only authorized users can read, modify, or delete it.

1. File Sharing Mechanisms

1.1 Multiple users and ownership model

Every file has an owner (the user who created it) and may be associated with a group of users. The system assigns a user ID (UID) and group ID (GID) to each process, and matches these against the file's owner and group information to decide what level of sharing is permitted for that process.

1.2 Sharing on a single system

- Multiple file names (links) can refer to the same underlying file/inode – hard links and symbolic (soft) links let several users access one physical file through different directory entries.
- The file system maintains a single copy of file metadata (size, timestamps, block pointers) so all sharers see the same underlying data.

1.3 Sharing across a network (distributed file systems)

In client-server file systems (e.g. NFS, AFS), a remote file appears to the user as if it were local, while the actual data resides on a server. Because multiple clients may cache and modify the same file, the system must define **consistency semantics** – a contract describing when a write made by one user becomes visible to other users:

- **UNIX semantics** – every write is visible to all other processes immediately (as if all users shared one open file). Simple and intuitive, but expensive to enforce over a network.
- **Session semantics** (used by AFS) – changes made by a process are visible only to that process until the file is closed; once closed, the new version becomes visible to sessions opened afterward. Reduces network traffic at the cost of stricter UNIX-style consistency.
- **Immutable shared-files semantics** – once a file is declared shared, it cannot be modified at all; a change simply creates a new file. This avoids the consistency problem entirely, at the cost of flexibility.

2. File Protection Mechanisms

Protection restricts the types of access that can be performed on a file – typically read, write, execute, append, delete, and list/rename – and to whom those operations are granted.

2.1 Access control lists (ACLs)

An ACL attached to a file lists specific users or groups together with the operations each is permitted to perform. This is the most flexible scheme, since arbitrary combinations of users and rights can be expressed, but a full ACL can become long and costly to store and search.

2.2 Owner / group / others (condensed access list)

To keep protection information compact, most systems (e.g. UNIX) condense users into three classes for every file – owner, group, and others – each with its own read/write/execute (rwx) bits. Special bits such as set-UID, set-GID, and the sticky bit refine this further for particular use cases.

2.3 Other mechanisms

- **Passwords** – a password can be attached to a file or directory; anyone presenting the correct password is granted access. Weak in practice since it grants all-or-nothing access and passwords must be remembered/shared.
- **Encryption** – file contents are stored in encrypted form; only holders of the decryption key can read the data, protecting it even if the underlying storage or backup is compromised.
- **Capability lists** – instead of the file storing who may access it, each user/process holds a "capability" (an unforgeable token) that itself grants specific rights to a specific file; access is checked by presenting the capability rather than looking up the file's list.

Access control matrix

Conceptually, sharing and protection together can be represented as an access control matrix, with files as columns, users/domains as rows, and each cell holding the set of operations that row is allowed on that column. ACLs are this matrix stored by column (per file); capability lists store it by row (per user).

User / Domain	payroll.txt	notes.txt	shared_doc.txt
Owner (Alice)	read, write, delete	read, write	read, write
Group (finance)	read	–	read, write
Others	–	read	read

3. Ensuring Security and Consistency Under Concurrent Access

3.1 Security – protection working alongside sharing

Whenever a process opens a shared file, the operating system checks the requesting UID/GID against the file's ACL or owner/group/others bits before granting a handle. Because this check happens on every open (and, in capability systems, on every operation), an unauthorized user cannot obtain access simply because a file is being shared by others – sharing widens who can reach the file, while protection continues to narrow what each of them can do to it. Encryption adds a second layer, protecting the data even if the access-control check itself is bypassed (e.g. via a stolen disk image).

3.2 Consistency – concurrency control mechanisms

Correct sharing also requires that simultaneous reads and writes do not corrupt the file or return stale/inconsistent data. Operating systems combine several mechanisms for this:

- **File/record locking** – a process can request a shared (read) lock, allowing other readers but no writers, or an exclusive (write) lock, blocking all other access to that file or byte-range until released.
- **Advisory vs mandatory locks** – advisory locks are only enforced if every cooperating process checks them (common in UNIX); mandatory locks are enforced by the kernel itself regardless of whether a process checks.
- **Consistency semantics** (UNIX / session / immutable-shared, discussed above) define precisely when a write becomes visible, preventing two users from unknowingly working from different versions of the same file.

- **Atomic operations** on metadata (e.g. atomic rename, atomic append) ensure that concurrent updates to directory structures or file length do not leave the file system in a half-updated state.

3.3 Putting it together

In practice, security and consistency are enforced as two cooperating layers: the protection layer (ACL / owner-group-others / capabilities, optionally with encryption) decides *who* is allowed to touch the file, while the concurrency-control layer (locking plus a defined consistency semantics) decides *how* those simultaneous, authorized accesses are serialized so the file's contents remain correct. A well-designed multi-user file system needs both: protection alone would still allow two authorized writers to corrupt a file by writing at the same time, and locking alone would still let an unauthorized user read or modify data they should never have reached.